Passing Array to Methods

Lecture 13

Based on Slides of Dr. Norazah Yusof

Passing Array Elements to a Method

- **When a single element of an array is passed to a method it is handled like any other variable.**
- **See example: PassElements.java**

Example 1

```
1. public class PassElements {
2.   public static void main (String[] args) {
3.     int[] numbers = {5, 10, 15, 20, 25, 30, 35, 40};
4.     for (int i=0; i<numbers.length; i++) {
5.       showValue(numbers[i]);
6.     }
7.     System.out.println();
8.   }
9.
10.  public static void showValue(int num) {
11.    System.out.print((num+2) + " ");
12.  }
13.}
```

Passing Arrays as Arguments to Methods

- **More often you will want to write methods to process array data by passing the entire array, not just one element at a time.**
- **An array can be passed as an argument to a method.**
- **To pass an array, you pass the value in the variable that references the array.**
- **See example: PassArray.java**

Example 2

```
1. import java.util.*;

2. public class PassArray {
3.   public static void main (String[] args) {
4.     final int SIZE = 4;
5.     int[] numbers = new int[SIZE];
6.     getValues(numbers);
7.     System.out.println("Here are the numbers.");
8.     showArray(numbers);
9.     System.out.println();
10.   }
```

Example 2

```
12. public static void getValues(int[] arr) {
13.     Scanner kb= new Scanner(System.in);
14.     System.out.println("Enter a series of " + arr.length +
15.         " numbers.");
16.     for (int i=0; i<arr.length; i++) {
17.         System.out.print("Enter number " + (i+1) + ":");
18.         arr[i] = kb.nextInt();
19.     }
20.   }
21. public static void showArray(int[] arr) {
22.     for (int i=0; i<arr.length; i++) {
23.         System.out.print(arr[i] + " ");
24.     }
25.   }
26.}
```

Pass By Value

- **Java uses *pass by value* to pass parameters to a method.**
- **There are important differences between passing a value of variables of primitive data types and passing arrays.**

Pass By Value

- **For a parameter of a primitive type value, the actual value is passed. Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.**
- **For a parameter of an array type, the value of the parameter contains a reference to an array; this reference is passed to the method. Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.**

Lab 5 – Exercise 1 – Question 1(i)

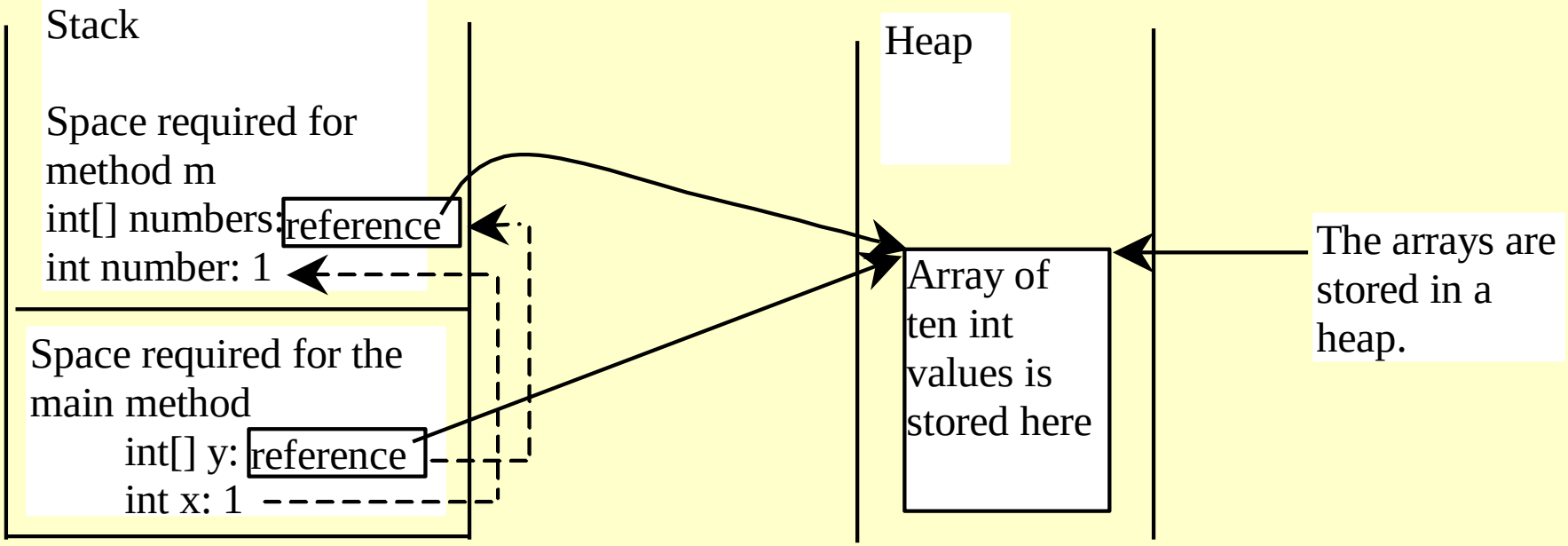
```
public class Lab7Ex1_1i {
  public static void main(String[] args) {
    int x = 1; // x represents an int value
    int[] y = new int[10]; // y represents an array of int values

    m(x, y); // Invoke m with arguments x and y

    System.out.println("x is " + x);
    System.out.println("y[0] is " + y[0]);
  }

  public static void m(int number, int[] numbers) {
    number = 1001; // Assign a new value to number
    numbers[0] = 5555; // Assign a new value to numbers[0]
  }
}
```

x is 1
y[0] is 5555

Heap

The Java Virtual Machine stores the array in an area of memory called the *heap*.

Call Stack and Heap



- **When invoking <u>m(x, y)</u>, the values of <u>x</u> and <u>y</u> are passed to <u>number</u> and <u>numbers</u>. Since <u>y</u> contains the reference value to the array, <u>numbers</u> now contains the same reference value to the same array.**
- **The JVM stores the array in an area of memory, called *heap*, which is used for dynamic memory allocation where blocks of memory are allocated and freed in an arbitrary order.**

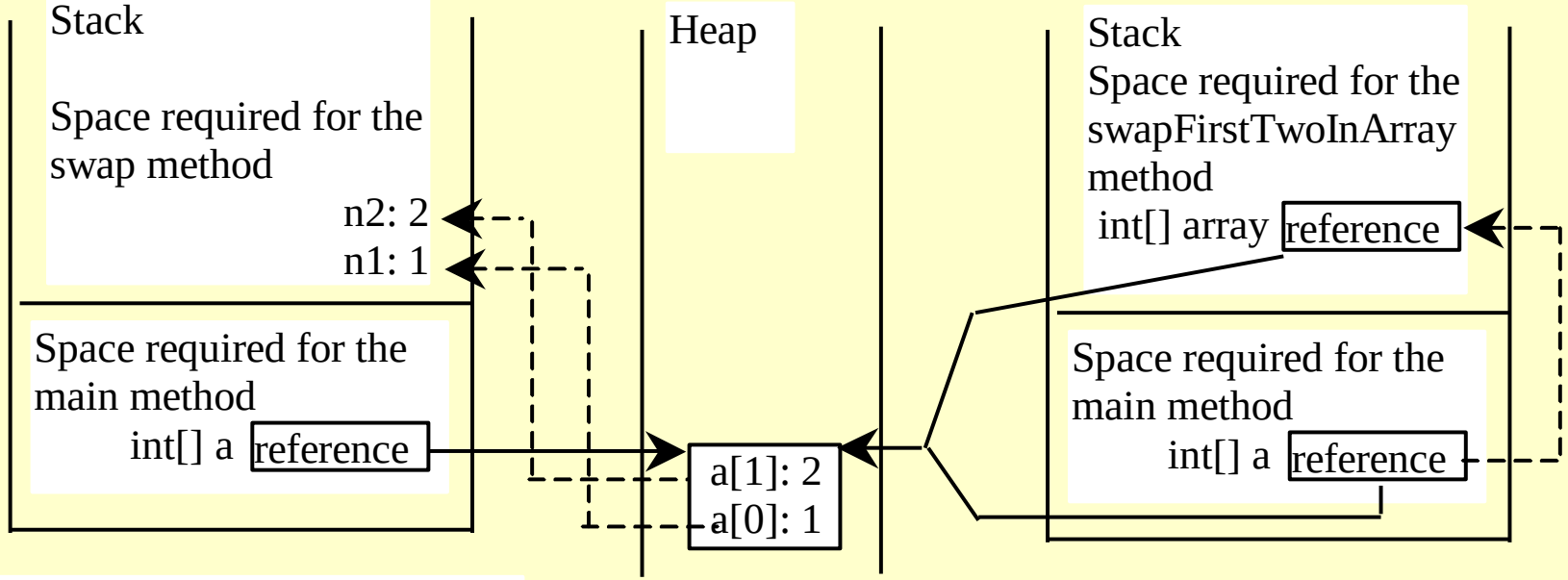
Passing Arrays as Arguments
Lab 5 – Exercise 1 – Question 1(ii)

```
public class Lab5a {
  public static void main(String[] args) {

    int[] a = {1,2}; // a represents an array of int values

    System.out.println("Before invoking swap");
    System.out.println("aray is {" + a[0] + ", " + a[1] + "}");
    swap(a[0],a[1]);
    System.out.println("After invoking swap");
    System.out.println("aray is {" + a[0] + ", " + a[1] + "}");

    System.out.println("Before invoking swapFirstTwoInArray");
   System.out.println("aray is {" + a[0] + ", " + a[1] + "}");
   swapFirstTwoInArray(a);
   System.out.println("After invoking swap");
    System.out.println("aray is {" + a[0] + ", " + a[1] + "}");
  }
```

Passing Arrays as Arguments
Lab 5 – Exercise 1 – Question 1(ii)

```
public static void swap(int n1, int n2) {
    int temp = n1;
    n1 = n2;
    n2 = temp;
  }

  public static void
  swapFirstTwoInArray(int[] array) {
      int temp = array[0];
      array[0] = array[1];
      array[1] = temp;
  }
}
```

Example, cont.



Invoke swap(int n1, int n2). The primitive type values in a[0] and a[1] are passed to the swap method.

The arrays are stored in a heap.

Invoke swapFirstTwoInArray(int[] array). The reference value in a is passed to the swapFirstTwoInArray method.

Returning an Array Reference
Lab 5 – Exercise 1 – Question 1(iii)

- A method can return a reference to an array.
- The return type of the method must be declared as an array of the right type.

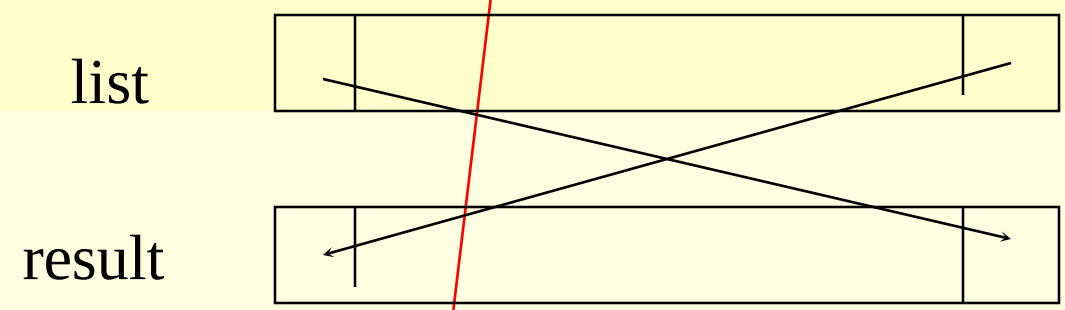
```
public static int[] reverse(int[] list) {
    int[] result = new int[list.length];
     for (int i = 0, j = result.length - 1;
         i < list.length; i++, j--) {
      result[j] = list[i];
    }
    return result;
```

- The `reverse()` method is a public static method that returns an array of int.

- **Create a new class**
- **Create a main method**
- **Create an array with size 5**
- **Print the values of the array**
- **Call method reserve and pass the array**
- **Again, print the values of the array**

Returning an Array from a Method
Lab 5 – Exercise 1 – Question 1(iii)

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];
  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }
  return result;
}
```




```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

Anonymous Array

The statement

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

creates an array using the following syntax:

```
new dataType[]{literal0, literal1, ..., literalk};
```

There is no explicit reference variable for the array. Such array is called an *anonymous array*.

Variable-Length Argument Lists

- **JDK 1.5 enables you to pass a variable number of arguments of the same type to a method.**
- **The parameter in the method is declared as follows:** `typeName ... parameterName`
- **Only one variable-length parameter may be specified in a method, and this parameter must be the last parameter.**
- **Java treats variable-length parameter as an array.**

Variable-Length Argument Lists
Lab 5 – Exercise 1 – Question 1(iv)

```
public class Lab7 {
  public static void main(String[] args) {

    printMax(21.8,32.2,13.0,45.2,25.8,36.5);
    printMax(new double[]{1,4,5,2});
  }

  public static void printMax(double ... numbers) {
    if (numbers.length == 0) {
      System.out.println("No argument passed");
      return;
    }

    double result = numbers[0];

    for (int i=1; i < numbers.length; i++)
        if (numbers[i] > result)
          result = numbers[i];

   System.out.println("The max value is " + result);
 }
}
```